DEEP LEARNING

Labwork 1

Students Ho Thanh Thuy Tien - 2540100

1 Implementation

In this labwork, I will create 3 function:

- f(x) : store the original function f(x)
- f_(x): store the derivative function of f(x)
- gradient_descent: the function to calculate gradient descent to find the minimum, stopping when the change in x fell below a specific threshold.

The gradient_descent is calculated by this rule:

$$x = x_A - L \cdot f'(x_A)$$

With:

- x: This is the new value (the updated position).
- x_A: This is the current value (the starting position for the current iteration).
- L: This is the Learning Rate
- $f'(x_A)$: The first-order derivative evaluated at the current position

2 Learning Rate Analysis

The learning rate L significantly affects the convergence speed and behavior of the algorithm. Based on my experiments, I observed the following:

- **Small Learning Rate ($L = 0.02$):** The algorithm is stable but requires a large number of iterations (over 148 steps) to reach the minimum.
- **Optimal Learning Rate ($L = 0.1$):** This rate provides the most efficient path, achieving convergence in only 36 steps with smooth progress.
- **Large Learning Rate ($L = 0.9$):** The large step size causes x to jump between positive and negative values, leading to unnecessary oscillations before stopping.

In conclusion, I found that choosing an appropriate learning rate is crucial: too small leads to inefficiency, while too large causes unnecessary oscillations. Therefore, in this case, 0.1 is the best option for learning rate